# Universidad Abierta Para Adultos

**ESCUELA**

**ingeniería y tecnología**

**ASIGNATURA**

Análisis y Diseño de sistema

**FACILITADOR**

Joah ML Gragorio Perez

**PARTICIPANTES**

Natanael Mateo - 100089414 (Lider, Organizador, Documentación)

Jordanny Santos – 100073059 (Diseñador)

Ana María Ferreira - 100087901 (Organizadora)

## Recinto

Santo Domingo/Santiago

**TEMA**

MediConnect - Hito 2

**FECHA**

12/04/2026

# PARTE I: ANÁLISIS INICIAL Y REQUERIMIENTOS (Hito 1)

## 1.1 Descripción del Proyecto

MediConnect es un sistema digital de gestión y reserva de citas médicas diseñado para modernizar y optimizar el proceso de agendamiento en clínicas y consultorios de salud. La plataforma ofrece una solución integral que conecta a pacientes, médicos, recepcionistas y administradores en un entorno digital unificado, eliminando las barreras del agendamiento telefónico tradicional y reduciendo la ineficiencia operativa inherente a los procesos manuales.

El sistema permite a los pacientes registrarse, buscar especialistas disponibles, reservar citas en línea, recibir recordatorios automáticos y gestionar sus citas desde cualquier dispositivo con acceso a internet. Para el personal médico y administrativo, MediConnect ofrece herramientas de gestión de agenda, control de sala de espera, generación de reportes gerenciales y validación de cobertura de seguros médicos en tiempo real.

## 1.2 Problema que Resuelve

### 1.2.1 Problemas del lado del paciente

- Largos tiempos de espera telefónica para agendar una cita, con frecuencia sin confirmación inmediata.
- Ausencia de recordatorios automáticos que lleva a altas tasas de inasistencia (no-shows).
- Imposibilidad de verificar la disponibilidad real del especialista sin llamar directamente a la clínica.
- Falta de historial centralizado que obliga al paciente a repetir su información médica en cada consulta.
- Dificultad para cancelar o reprogramar citas fuera del horario de atención de la clínica.

### 1.2.2 Problemas del lado institucional

- Pérdida de ingresos por citas no atendidas que no liberan el cupo a tiempo para otros pacientes.
- Procesos manuales de agendamiento propensos a errores humanos como doble reserva o información incorrecta.
- Ausencia de datos consolidados para la toma de decisiones gerenciales sobre capacidad, demanda y rendimiento.
- Ineficiencia en la validación de cobertura de seguros médicos, que en muchos casos se hace manualmente.

## 1.3 Usuarios Objetivo

| Tipo de Usuario | Perfil | Necesidades Principales |
|---|---|---|
| Paciente | Persona mayor de 16 años con necesidad de atención médica | Registrarse, buscar especialistas, reservar, cancelar y recibir recordatorios de citas |

| Tipo de Usuario | Perfil | Necesidades Principales |
|---|---|---|
| Médico / Especialista | Profesional de la salud que ofrece consultas en la clínica | Gestionar agenda, visualizar citas del día, recibir notificaciones y bloquear horarios no disponibles |
| Recepcionista | Personal administrativo de la clínica | Gestionar el flujo de sala de espera, verificar citas mediante QR, registrar llegadas |
| Gerente / Administrador | Directivo responsable de la operación de la clínica | Generar reportes estadísticos, monitorear indicadores, administrar usuarios |
| Administrador TI | Responsable técnico del sistema | Configurar el sistema, gestionar integraciones, administrar roles y seguridad |
| Sistema de Aseguradoras | Sistema externo de validación de seguros | Recibir solicitudes de verificación de cobertura y responder en tiempo real |

## 1.4 Propuesta de Valor

MediConnect ofrece una propuesta de valor diferenciada que beneficia simultáneamente a los pacientes y a las instituciones de salud, posicionándose como la solución más completa, accesible y eficiente para la gestión de citas médicas en la República Dominicana.

**Para los pacientes:**

- Acceso al sistema de reservas en cualquier momento y desde cualquier dispositivo.
- Recepción instantánea de un comprobante digital con código QR único al momento de confirmar la reserva.
- Notificaciones automáticas vía SMS y correo electrónico 24 horas y 2 horas antes de la cita.
- Historial médico centralizado accesible desde el portal del paciente.

**Para la institución:**

- Reducción estimada del 40% en la tasa de inasistencias (no-shows) mediante recordatorios automatizados.
- Eliminación de doble reserva mediante verificación de disponibilidad en tiempo real.
- Dashboard gerencial con KPIs actualizados en tiempo real: ocupación, ingresos y pacientes frecuentes.

## 1.5 Alcance del Sistema

| Característica | Descripción Funcional | Unidad |
|---|---|---|
| Gestión de citas | Agenda de citas por médico, fecha y hora, con confirmación | III |
| Historial clínico | Registro de consultas, diagnósticos, recetas y resultados | IV, IX |
| Módulo de pacientes | Registro y gestión de datos personales e historial médico | III |
| Facturación | Generación de facturas por consultas y servicios adicionales | V |

| Característica | Descripción Funcional | Unidad |
|---|---|---|
| Reportes estadísticos | Consultas por especialidad, ingresos mensuales, pacientes frecuentes | VI |
| Prototipo de interfaz | Pantallas de agenda, historial clínico, facturación | VIII |
| Modelo de datos | Entidades: Paciente, Médico, Cita, Consulta, Factura, Especialidad | IV |

## 1.6 Restricciones y Suposiciones

**Restricciones:**

- El sistema debe ser accesible desde navegadores modernos (Chrome, Firefox, Edge, Safari) sin instalación adicional.
- El tiempo de respuesta máximo para operaciones críticas (reserva de cita) no debe superar 3 segundos.
- El sistema debe soportar al menos 100 usuarios concurrentes en la versión inicial.
- Todos los datos de pacientes deben ser almacenados en servidores ubicados en territorio dominicano o con cumplimiento de la Ley 172-13 de protección de datos personales.

**Suposiciones:**

- La clínica cuenta con conectividad a internet estable en todas sus instalaciones.
- Los pacientes tienen acceso a un dispositivo con conexión a internet para realizar reservas en línea.
- El personal médico y administrativo recibirá capacitación básica antes del lanzamiento del sistema.

## 1.7 Beneficios Esperados

| Beneficio | Indicador de Éxito | Plazo |
|---|---|---|
| Reducción de inasistencias | Tasa de no-show < 15% (actualmente ~35%) | 3 meses post-lanzamiento |
| Aumento de eficiencia administrativa | Reducción del 60% en tiempo de gestión manual de citas | 6 meses post-lanzamiento |
| Incremento de satisfacción del paciente | NPS ≥ 70 en encuesta post-consulta | 6 meses post-lanzamiento |
| Optimización de ingresos | Incremento del 20% en consultas atendidas por disponibilidad de agenda | 12 meses post-lanzamiento |

## 1.8 Factores Críticos de Éxito

- Adopción por parte del personal médico y administrativo desde el primer día de operación.
- Integración exitosa con el sistema de aseguradoras para validación de coberturas en tiempo real.
- Experiencia de usuario intuitiva que no requiera capacitación extensa para pacientes.

- Disponibilidad del sistema del 99.5% en horario de operación de la clínica.

## 1.9 Visión a Futuro

- Telemedicina integrada: módulo de consultas virtuales con videollamada directamente desde la plataforma.
- Integración con laboratorios: resultados de exámenes disponibles automáticamente en el historial del paciente.
- Aplicación móvil nativa (iOS y Android) con funcionalidades adicionales como check-in por GPS.
- Inteligencia artificial para sugerencia de especialista según síntomas reportados por el paciente.
- Expansión a múltiples clínicas con modelo multi-tenant para cadenas de centros médicos.

# PARTE II: MODELADO DE PROCESOS Y DATOS (Hito 2)

## 2.1 Diagramas de Casos de Uso

### 2.1.1 Módulo de Gestión de Citas

| Elemento UML | Descripción |
|---|---|
| Actor: Paciente | Solicitar cita, cancelar cita, ver historial de citas |
| Actor: Recepcionista | Registrar cita, confirmar cita, reagendar cita, cancelar cita |
| Actor: Médico | Ver agenda del día, confirmar disponibilidad, registrar atención |
| UC-01: Solicitar Cita | Paciente selecciona médico, especialidad, fecha y hora disponible |
| UC-02: Confirmar Cita | Sistema envía confirmación al paciente por correo/SMS |
| UC-03: Cancelar Cita | Paciente o Recepcionista cancela con al menos 24h de anticipación |
| UC-04: Ver Agenda | Médico visualiza citas del día, semana o mes |
| UC-05: Reagendar Cita | Recepcionista modifica fecha/hora a solicitud del paciente |

### 2.1.2 Historial Clínico

| Elemento UML | Descripción |
|---|---|
| Actor: Médico | Crear consulta, registrar diagnóstico, emitir receta, ver historial |
| Actor: Paciente | Ver su propio historial clínico, solicitar copia de resultados |
| Actor: Admin | Gestionar acceso a registros, generar reportes clínicos |
| UC-06: Registrar Consulta | Médico documenta la atención: síntomas, diagnóstico, tratamiento |
| UC-07: Emitir Receta | Médico genera prescripción médica asociada a consulta |
| UC-08: Ver Historial | Consulta de antecedentes, diagnósticos y recetas del paciente |
| UC-09: Adjuntar Resultados | Admin o médico sube resultados de laboratorio/imágenes |

### 2.1.3 Módulo de Facturación

| Elemento UML | Descripción |
|---|---|
| Actor: Recepcionista/Admin | Generar factura, registrar pago, emitir recibo |
| Actor: Paciente | Realizar pago, solicitar factura fiscal, ver estado de cuenta |
| UC-10: Generar Factura | Sistema crea factura automática al finalizar consulta |
| UC-11: Registrar Pago | Admin registra pago: efectivo, tarjeta, seguro médico |
| UC-12: Generar Reporte | Gerencia solicita reporte de ingresos por período |

## 2.2 Diagramas de Actividades

### 2.2.1 Proceso de Agendamiento de Cita

| Paso | Paciente | Sistema | Médico/Admin |
|---|---|---|---|
| 1 | Accede al portal web/app | | |
| 2 | Selecciona especialidad y médico | | |
| 3 | | Muestra disponibilidad de horarios | |
| 4 | Elige fecha y hora disponible | | |
| 5 | | Verifica disponibilidad y crea cita | |
| 6 | | Envía confirmación por email/SMS | Recibe notificación de nueva cita |
| 7 | Recibe confirmación con código de cita | | |

### 2.2.2 Proceso de Consulta Médica

- Inicio: Paciente llega a la clínica y se registra en recepción.
- Verificación: Recepcionista confirma cita en el sistema.
- Sala de espera: Paciente aguarda llamado del médico.
- Atención: Médico atiende al paciente, registra síntomas y diagnóstico.
- [Decisión] ¿Requiere exámenes adicionales?
  - SÍ: Se solicitan exámenes, se agenda nueva cita de seguimiento.
  - NO: Médico emite receta y cierra consulta.
- Facturación: Se genera factura automáticamente.
- Pago: Paciente realiza pago en caja.
- Fin: Paciente recibe recibo y copia de receta.

### 2.2.3 Proceso de Facturación

- Inicio: Cierre de consulta médica en el sistema.
- Cálculo: Sistema calcula costo (consulta + servicios adicionales).
- [Decisión] ¿Paciente tiene seguro médico?
  - SÍ: Sistema calcula copago según cobertura del seguro.
  - NO: Monto total a cargo del paciente.
- Generación: Sistema genera factura con detalles completos.
- [Decisión] ¿Tipo de pago? Efectivo / Tarjeta / Seguro.
- Cierre: Sistema actualiza estado de factura a 'Pagada'.
- Fin: Se emite recibo y se archiva en historial del paciente.

## 2.3 Diagramas de Secuencia

### 2.3.1 Solicitud y Confirmación de Cita

| Paso | Paciente (Actor) | :SistemaCitas | :AgendaService | :NotificacionSrv |
|---|---|---|---|---|
| 1 | solicitarCita(médico, fecha, hora) | | | |
| 2 | | verificarDisponibilidad(médico, fecha) | | |
| 3 | | | consultarHorario(médico) | |
| 4 | | | return: disponible = true | |
| 5 | | crearCita(paciente, médico, fecha) | | |
| 6 | | enviarConfirmacion(paciente, cita) | | |
| 7 | | | | notificar(email/SMS) |
| 8 | return: confirmacion + codigoCita | | | |

### 2.3.2 Registro de Consulta Médica

| Paso | Médico (Actor) | :SistemaConsulta | :HistorialService | :FacturaService |
|---|---|---|---|---|
| 1 | abrirConsulta(citaId) | | | |
| 2 | | cargarPaciente(pacienteId) | | |

| Paso | Médico (Actor) | :SistemaConsulta | :HistorialService | :FacturaService |
|---|---|---|---|---|
| 3 | | | getHistorial(pacienteId) | |
| 4 | registrarDiagnostico(datos) | | | |
| 5 | emitirReceta(medicamentos) | | | |
| 6 | cerrarConsulta() | | guardarConsulta() | |
| 7 | | generarFactura(consulta) | | crearFactura(monto) |

## 2.4 Especificaciones de Casos de Uso

### CU-01: Solicitar Cita Médica

| Campo | Detalle |
|---|---|
| Identificador | CU-01 |
| Actor Principal | Paciente |
| Precondiciones | El paciente está registrado en el sistema. El médico seleccionado tiene horarios disponibles. |
| Postcondiciones | La cita queda registrada con estado 'Confirmada'. Se envía notificación al paciente y al médico. |
| Flujo Principal | 1. Paciente accede al módulo de citas. 2. Sistema muestra especialidades disponibles. 3. Paciente selecciona especialidad y médico. 4. Sistema muestra calendario con horarios disponibles. 5. Paciente selecciona fecha y hora. 6. Sistema verifica disponibilidad. 7. Paciente confirma datos. 8. Sistema registra cita. 9. Sistema envía correo de confirmación con código. |
| Flujos Alternativos | 6a. Si horario no disponible: el sistema muestra mensaje y regresa al paso 4. 7a. Si paciente cancela: descarta datos y regresa a pantalla principal. |
| Excepciones | Sistema no disponible: se muestra mensaje de error e invita a intentarlo más tarde. |

### CU-02: Registrar Consulta Médica

| Campo | Detalle |
|---|---|
| Identificador | CU-02 |
| Actor Principal | Médico |
| Precondiciones | El médico ha iniciado sesión. Existe una cita confirmada para la fecha y hora actuales. |

| Campo | Detalle |
|---|---|
| Postcondiciones | La consulta queda registrada en el historial clínico del paciente. Se genera automáticamente una factura. |
| Flujo Principal | 1. Médico selecciona cita activa. 2. Sistema carga datos e historial previo. 3. Médico registra síntomas. 4. Médico registra diagnóstico (código CIE-10). 5. Médico indica tratamiento. 6. Médico emite receta si corresponde. 7. Médico cierra consulta. 8. Sistema guarda consulta en historial. 9. Sistema genera factura automáticamente. |
| Flujos Alternativos | 6a. No se requiere receta: médico omite paso 6. 4a. Médico solicita exámenes: se registra orden y se puede agendar seguimiento. |
| Excepciones | Pérdida de conexión: sistema guarda automáticamente los datos ingresados hasta el momento. |

### CU-04: Generar Factura

| Campo | Detalle |
|---|---|
| Identificador | CU-04 |
| Actor Principal | Sistema / Recepcionista |
| Precondiciones | Existe una consulta cerrada sin factura asociada. |
| Postcondiciones | La factura queda en estado 'Pendiente de pago' y se asocia al paciente y consulta correspondiente. |
| Flujo Principal | 1. Al cierre de consulta, el sistema inicia la generación de factura. 2. Sistema obtiene servicios realizados. 3. Sistema calcula costo total según tarifario. 4. Sistema verifica si paciente tiene seguro activo. 5. Si tiene seguro: calcula copago. 6. Sistema genera factura con número único. 7. Factura queda en estado 'Pendiente de pago'. 8. Recepcionista notifica al paciente. |
| Flujos Alternativos | 4a. Sin seguro: el monto total es asumido por el paciente. 2a. Servicios adicionales no registrados: Recepcionista puede agregar ítems manualmente. |
| Excepciones | Error en cálculo de tarifas: sistema alerta a la Recepcionista para revisión manual. |

## 2.5 Modelo Entidad-Relación

### Catálogo de Entidades Principales

| Entidad | Atributos Principales (PK / FK) | Relaciones |
|---|---|---|
| PACIENTE | paciente_id (PK), num_documento, nombre, apellido, fecha_nacimiento, genero, telefono, email, estado | Tiene Citas, Consultas, Facturas, HistorialClínico |
| MEDICO | medico_id (PK), num_colegiado, nombre, especialidad_id (FK), consultorio_id (FK), estado | Atiende Citas, registra Consultas |

| Entidad | Atributos Principales (PK / FK) | Relaciones |
|---|---|---|
| ESPECIALIDAD | especialidad_id (PK), nombre, duracion_cita_min, tarifa_base, estado | Clasifica a Médicos |
| CITA | cita_id (PK), paciente_id (FK), medico_id (FK), fecha_cita, hora_inicio, hora_fin, estado, codigo_confirmacion | Genera una Consulta |
| CONSULTA | consulta_id (PK), cita_id (FK), paciente_id (FK), medico_id (FK), diagnostico, codigo_CIE10, tratamiento, estado | Genera Recetas y Facturas |
| FACTURA | factura_id (PK), consulta_id (FK), paciente_id (FK), subtotal, descuento, total, estado_pago, fecha_emision | Registra Pagos |
| RECETA | receta_id (PK), consulta_id (FK), medico_id (FK), paciente_id (FK), indicaciones, estado, vigencia | Contiene Detalles de Medicamentos |
| PAGO | pago_id (PK), factura_id (FK), monto, tipo_pago, fecha_pago, referencia | Asociado a Factura |

## Relaciones entre Entidades

| Relación | Tipo | Multiplicidad | Descripción |
|---|---|---|---|
| Paciente → Cita | Asociación | 1..* | Un paciente tiene una o más citas agendadas |
| Médico → Cita | Asociación | 1..* | Un médico atiende múltiples citas |
| Cita → Consulta | Composición | 0..1 | Una cita puede originar una consulta |
| Consulta → Receta | Composición | 0..* | Una consulta puede generar múltiples recetas |
| Consulta → Factura | Composición | 1 | Cada consulta cerrada genera exactamente una factura |
| Factura → Pago | Composición | 1..* | Una factura puede tener uno o más pagos |
| Médico → Especialidad | Asociación | *..1 | Varios médicos pertenecen a una especialidad |
| Receta → DetalleReceta | Composición | 1..* | Una receta contiene uno o más medicamentos |

## Nota sobre Normalización

El modelo de datos cumple con la Tercera Forma Normal (3FN). Todas las dependencias transitivas han sido eliminadas y cada atributo no clave depende únicamente de la clave primaria de su entidad. Se implementan 20 entidades en total, incluyendo tablas intermedias para relaciones muchos-a-muchos y catálogos de soporte.

# PARTE III: DISEÑO TÉCNICO Y PROTOTIPO (Hito 3)

## 3.1 Diagrama de Clases

### 3.1.1 Capa de Dominio — Clases Principales

| Clase | Atributos | Métodos |
|---|---|---|
| Paciente | -pacienteId: int -numDocumento: String -nombre: String -apellido: String -fechaNacimiento: Date -email: String -telefono: String -estado: EstadoPaciente | +getDatosCompletos(): String +calcularEdad(): int +tieneSeguroActivo(): boolean +getCitasPendientes(): List<Cita> |
| Medico | -medicoId: int -numColegiado: String -nombre: String -especialidad: Especialidad -consultorio: Consultorio -estado: EstadoMedico | +getAgendaDia(fecha: Date): List<Cita> +getAgendaSemana(fecha: Date): List<Cita> +verificarDisponibilidad(fecha, hora): boolean +getCitasHoy(): List<Cita> |
| Cita | -citaId: int -paciente: Paciente -medico: Medico -fechaCita: Date -horaInicio: Time -horaFin: Time -estado: EstadoCita -motivoConsulta: String | +confirmar(): void +cancelar(motivo: String): void +reagendar(nuevaFecha, nuevaHora): void +generarCodigoConfirmacion(): String +notificarPaciente(): void |
| Consulta | -consultaId: int -cita: Cita -paciente: Paciente -medico: Medico -diagnosticoPrincipal: String -codigoCIE10: String -tratamiento: String | +abrir(): void +cerrar(): void +agregarDiagnostico(d: Diagnostico): void +generarReceta(): Receta +generarFactura(): Factura |
| Factura | -facturaId: int -numeroFactura: String -consulta: Consulta -paciente: Paciente -subtotal: BigDecimal -total: BigDecimal -estado: EstadoFactura | +calcularTotal(): BigDecimal +aplicarDescuento(pct: double): void +aplicarSeguro(seguro: SeguroMedico): void +registrarPago(pago: Pago): void +generarPDF(): byte[] |
| Receta | -recetaId: int -numeroReceta: String -consulta: Consulta -medico: Medico -paciente: Paciente -detalles: List<DetalleReceta> -indicaciones: String | +agregarMedicamento(d: DetalleReceta): void +validar(): boolean +generarPDF(): byte[] +isVigente(): boolean |

### 3.1.2 Enumeraciones del Sistema

| EstadoCita | EstadoConsulta | EstadoFactura | EstadoPaciente |
|---|---|---|---|
| SOLICITADA | EN_CURSO | PENDIENTE | ACTIVO |
| CONFIRMADA | COMPLETADA | PAGADA | INACTIVO |
| CANCELADA | CANCELADA | PARCIAL | SUSPENDIDO |
| COMPLETADA | | ANULADA | |
| NO_ASISTIO | | | |

## 3.2 Diagrama de Componentes

| Componente | Responsabilidad | Interfaces Provistas / Requeridas |
|---|---|---|
| Frontend Web (React) | Interfaz de usuario. Renderiza vistas para paciente, médico y administrador. Gestiona estado local y comunicación con API. | Provee: UI para todos los roles. Requiere: API REST del Backend |
| API Gateway | Punto único de entrada. Maneja autenticación JWT, rate limiting, enrutamiento de peticiones. | Provee: API REST unificada. Requiere: Auth Service, servicios internos |
| Auth Service | Gestión de identidad y acceso. Login, logout, renovación de tokens JWT, manejo de roles y permisos. | Provee: /auth/login, /auth/refresh. Requiere: Base de datos de usuarios |
| Citas Service | Lógica de negocio para gestión de citas: creación, confirmación, cancelación, reagendamiento y disponibilidad. | Provee: /api/citas. Requiere: DB, Notification Service |
| Consultas Service | Registro y gestión de consultas médicas, diagnósticos, recetas y resultados de exámenes. | Provee: /api/consultas, /api/recetas. Requiere: DB, Factura Service |
| Facturación Service | Generación de facturas, cálculo de totales, aplicación de seguros y registro de pagos. | Provee: /api/facturas, /api/pagos. Requiere: DB, PDF Service |
| Reportes Service | Generación de reportes estadísticos: citas por especialidad, ingresos mensuales, pacientes frecuentes. | Provee: /api/reportes. Requiere: DB (solo lectura) |
| Notification Service | Envío de notificaciones por email (SMTP) y SMS (Twilio). Confirmaciones de citas, recordatorios y alertas. | Provee: Interfaz de notificaciones. Requiere: SMTP Server, Twilio API |
| PDF Service | Generación de documentos PDF: facturas, recetas médicas y reportes. | Provee: Generación de PDF. Requiere: Plantillas HTML |
| Base de Datos (PostgreSQL) | Almacenamiento persistente de todos los datos del sistema con soporte transaccional completo. | Provee: Conexión JDBC/ODBC. Requiere: — |

## 3.3 Diagrama de Despliegue

### Descripción de Nodos de Infraestructura

| Nodo | Tecnología | Responsabilidad |
|---|---|---|
| Cliente Web | Chrome/Firefox/Edge | Navegador del usuario final (paciente, médico, admin) |
| CDN / Load Balancer | Nginx / Cloudflare | Distribución de tráfico y caché de activos estáticos |

| Nodo | Tecnología | Responsabilidad |
|---|---|---|
| Servidor de Aplicación | Node.js + Express (Docker) | Lógica de negocio, API REST, autenticación JWT |
| Servidor de Base de Datos | PostgreSQL 15 (Docker) | Almacenamiento relacional principal del sistema |
| Servidor de Caché | Redis 7 (Docker) | Caché de sesiones, refresh tokens y reportes frecuentes |
| Servidor de Archivos | S3-compatible (MinIO) | Almacenamiento de PDFs, imágenes y documentos adjuntos |
| Servidor SMTP | SendGrid API | Envío de correos de confirmación y recordatorios |
| API SMS | Twilio API | Envío de SMS de recordatorio y confirmación de citas |

## 3.4 API REST — Endpoints Principales

| Método | Endpoint | Descripción | Auth Requerida |
|---|---|---|---|
| POST | /auth/login | Autenticación del usuario, retorna JWT | No |
| POST | /auth/logout | Invalida el refresh token en Redis | JWT Bearer |
| GET | /api/citas | Lista citas con filtros (fecha, médico, paciente, estado) | JWT + Rol |
| POST | /api/citas | Crear nueva cita — verifica disponibilidad | JWT + Rol |
| PATCH | /api/citas/:id/estado | Confirmar, cancelar o completar una cita | JWT + Rol |
| GET | /api/pacientes/:id | Obtener ficha completa del paciente | JWT + Rol |
| POST | /api/consultas | Registrar nueva consulta médica | JWT + MEDICO |
| POST | /api/consultas/:id/cerrar | Cerrar consulta — genera factura automáticamente | JWT + MEDICO |
| GET | /api/pacientes/:id/historial | Historial clínico completo del paciente | JWT + Rol |
| POST | /api/facturas/:id/pagos | Registrar pago parcial o total de factura | JWT + ADMIN |
| GET | /api/facturas/:id/pdf | Generar y descargar PDF de la factura | JWT + Rol |

| Método | Endpoint | Descripción | Auth Requerida |
|---|---|---|---|
| GET | /api/reportes/citas | Reporte de citas por especialidad y período | JWT + ADMIN |
| GET | /api/reportes/ingresos | Reporte de ingresos mensuales | JWT + ADMIN |

## 3.5 Análisis de Costos y ROI

### 3.5.1 Inversión Inicial de Desarrollo

| Rubro | Descripción | Costo (USD) |
|---|---|---|
| Desarrollo Backend | API REST, servicios, base de datos (480 hrs) | $14,400 |
| Desarrollo Frontend | React, UI/UX, componentes (320 hrs) | $9,600 |
| Diseño UI/UX | Prototipo, wireframes, sistema de diseño (80 hrs) | $2,400 |
| Infraestructura inicial | Servidores, dominios, certificados SSL | $2,400 |
| QA y pruebas | Testing unitario, integración, E2E (80 hrs) | $2,400 |
| Documentación técnica | Manuales de usuario y técnicos | $100 |
| Total Inversión | | $31,300 |

### 3.5.2 ROI y Análisis Financiero (3 años)

| Métrica Financiera | Año 1 (USD) | Año 2 (USD) | Año 3 (USD) |
|---|---|---|---|
| Inversión inicial (desarrollo) | ($31,300) | — | — |
| Costos operativos (infra + mantenimiento) | ($14,580) | ($15,600) | ($15,600) |
| Beneficios totales | $72,000 | $72,000 | $72,000 |
| Flujo neto del período | $26,120 | $56,400 | $56,400 |
| Flujo acumulado | $26,120 | $82,520 | $138,920 |
| ROI del período (%) | 38.1% | 82.4% | 126.5% |

| Indicador | Valor |
|---|---|
| Inversión total (3 años) | USD $76,780 |
| Beneficios totales (3 años) | USD $216,000 |
| Ganancia neta (3 años) | USD $139,220 |

| Indicador | Valor |
|---|---|
| ROI Global (3 años) | 181.3% |
| Período de recuperación (Payback) | Aproximadamente 7 meses desde el lanzamiento |
| Valor Presente Neto (VPN al 10%) | USD $108,450 — proyecto financieramente viable |

## 3.6 Decisiones de Diseño Arquitectónico

### Decisión 1: Arquitectura Monolítica Modular vs Microservicios

**Decisión tomada: Arquitectura Monolítica Modular.**

Para el tamaño actual del equipo (2-3 desarrolladores) y la escala de la clínica, una arquitectura monolítica modular es más apropiada. Los microservicios añaden complejidad operacional (orquestación, comunicación entre servicios, monitoreo distribuido) que no se justifica en esta etapa. Los módulos están desacoplados mediante interfaces claras, permitiendo una migración futura a microservicios si el sistema crece.

### Decisión 2: Base de Datos Relacional (PostgreSQL 15)

**Decisión tomada: PostgreSQL 15 (relacional).**

Los datos médicos son altamente relacionales: citas relacionan pacientes, médicos, especialidades y consultorios. Las consultas médicas generan diagnósticos, recetas y facturas con relaciones estrictas de integridad referencial. Una base de datos relacional con soporte ACID completo garantiza la consistencia de los datos clínicos y financieros.

### Decisión 3: Autenticación JWT con Redis

**Decisión tomada: JWT stateless con refresh token almacenado en Redis.**

Los tokens JWT permiten validación sin consultar la base de datos en cada petición, reduciendo la latencia. El refresh token en Redis permite invalidar sesiones de forma centralizada (logout real). Access token: 8 horas de validez. Refresh token: 7 días en Redis con TTL.

## 3.7 Plan de Pruebas

| Tipo de Prueba | Herramienta | Cobertura Objetivo | Casos Clave |
|---|---|---|---|
| Unitarias | Jest | 70% lógica de negocio | CitaService.crearCita(), FacturaService.calcularTotal(), validaciones de disponibilidad |
| Integración | Jest + Supertest | Endpoints críticos de API | POST /api/citas, POST /auth/login, POST /api/consultas/:id/cerrar |
| Componentes React | React Testing Library | Componentes críticos de formularios | Formulario de cita, modal de pago, formulario de consulta |
| E2E | Cypress | Flujos principales completos | Flujo completo: login → agendar cita → consulta → factura → pago |
| Carga / Performance | Artillery.io | API bajo carga concurrente | 50 usuarios concurrentes, GET /api/citas — tiempo de respuesta < 500ms |

## 3.8 Gestión de Riesgos Técnicos

| Riesgo | Probabilidad | Impacto | Mitigación |
|---|---|---|---|
| Pérdida de datos por fallo de BD | Baja | Crítico | Backups automáticos diarios a S3, réplica de BD en standby |
| Brecha de seguridad en datos médicos | Media | Alto | Encriptación AES-256, auditoría de accesos, pruebas de penetración antes del lanzamiento |
| Tiempo de respuesta lento en reportes | Alta | Medio | Índices en BD, caché de reportes frecuentes en Redis, generación asíncrona |
| Dependencia de terceros (Twilio, SMTP) | Media | Bajo | Colas de reintento, fallback a solo email si SMS falla |
| Baja adopción por usuarios finales | Media | Alto | Capacitación presencial, manual de usuario, soporte en las primeras 4 semanas |

# PARTE IV: INTEGRACIÓN FINAL Y SÍNTESIS DEL PROYECTO (Hito 4)

## 4.1 Resumen Ejecutivo del Proyecto

MediConnect es un sistema digital integral de gestión de clínica médica desarrollado como proyecto académico para la asignatura de Análisis y Diseño de Sistemas. A lo largo de cuatro hitos, el proyecto ha evolucionado desde la concepción inicial hasta un diseño técnico completo, listo para su implementación.

El sistema aborda una problemática real y relevante del sector salud en la República Dominicana: la ineficiencia en el proceso de agendamiento y gestión de citas médicas en clínicas privadas. La solución propuesta integra gestión de citas, historial clínico, facturación y reportes estadísticos en una plataforma unificada.

## 4.2 Integración de los Ocho Módulos del Sistema

| Unidad | Módulo | Estado | Hito |
|---|---|---|---|
| III | Gestión de Citas y Módulo de Pacientes | Completo | Hito 2 |
| IV | Modelo de Datos (MER) e Historial Clínico | Completo | Hito 2 |
| V | Facturación y Registro de Pagos | Completo | Hito 2 |
| VI | Reportes Estadísticos y Dashboard Gerencial | Completo | Hito 2 |
| VIII | Prototipo de Interfaz (Agenda, Historial, Facturación) | Completo | Hito 3 |
| IV (Avanzado) | Diagrama de Clases UML | Completo | Hito 3 |
| IV (Avanzado) | Diagrama de Componentes y Despliegue | Completo | Hito 3 |
| IX | Diagramas de Secuencia y Casos de Uso Extendidos | Completo | Hito 2 |

## 4.3 Arquitectura Final del Sistema

### Visión General de Capas

- Capa de Presentación: Aplicación React con componentes reutilizables, responsive design y accesibilidad WCAG 2.1 AA.
- Capa de API: API Gateway centralizado con autenticación JWT, rate limiting y enrutamiento a servicios.
- Capa de Servicios: Módulos de negocio independientes (Citas, Consultas, Facturación, Reportes, Notificaciones).
- Capa de Datos: PostgreSQL 15 con esquema normalizado (3FN), Redis para caché y gestión de sesiones.

- Capa de Infraestructura: Docker containers en servidor cloud, CDN para activos estáticos, S3 para documentos.

### Flujo de Datos Principal

| Paso | Actor | Acción | Sistema Involucrado |
|---|---|---|---|
| 1 | Paciente | Accede al portal y solicita cita | Frontend React → API Gateway |
| 2 | Sistema | Verifica disponibilidad del médico | Citas Service → PostgreSQL |
| 3 | Sistema | Registra cita y envía confirmación | Citas Service → Notification Service → Twilio/SMTP |
| 4 | Médico | Atiende al paciente y registra consulta | Consultas Service → HistorialService → PostgreSQL |
| 5 | Sistema | Genera factura automáticamente al cerrar consulta | Consultas Service → Facturación Service → PDF Service |
| 6 | Recepcionista | Registra pago del paciente | Facturación Service → PostgreSQL → Notificación recibo |
| 7 | Gerencia | Consulta reportes estadísticos del período | Reportes Service → PostgreSQL (solo lectura) → Redis caché |

## 4.4 Estándares de Desarrollo Implementados

| Área | Estándar / Convención |
|---|---|
| Nombrado de variables | camelCase para variables y funciones, PascalCase para clases y componentes React, UPPER_SNAKE_CASE para constantes |
| Estructura Backend | /src/controllers, /src/services, /src/repositories, /src/models, /src/middlewares, /src/utils, /src/routes |
| Estructura Frontend | /src/components, /src/pages, /src/hooks, /src/context, /src/services (API calls), /src/utils, /src/assets |
| Respuestas API REST | Formato estándar: { success: boolean, data: object\|array, message: string, errors: [] }. Códigos HTTP: 200, 201, 400, 401, 403, 404, 422, 500 |
| Commits Git | Conventional Commits: feat:, fix:, docs:, refactor:, test:, chore: — Ramas: main, develop, feat/nombre-feature |
| Documentación API | Swagger / OpenAPI 3.0 — Disponible en /api/docs en ambiente de desarrollo |
| Pruebas | Jest para unit tests (backend), React Testing Library para componentes. Cobertura mínima objetivo: 70% en lógica de negocio |
| Linting y formato | ESLint con reglas Airbnb + Prettier para formateo automático. Husky para pre-commit hooks |

## 4.5 Modelo de Datos — Script SQL Principal

*A continuación se presenta el script SQL de creación de las tablas principales del sistema:*

```
CREATE TABLE especialidad (
  especialidad_id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  duracion_cita_min INT DEFAULT 30,
  tarifa_base DECIMAL(10,2) NOT NULL,
  estado VARCHAR(10) DEFAULT 'ACTIVO'
);

CREATE TABLE paciente (
  paciente_id SERIAL PRIMARY KEY,
  num_documento VARCHAR(20) NOT NULL UNIQUE,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  email VARCHAR(150) UNIQUE,
  telefono VARCHAR(20),
  estado VARCHAR(10) DEFAULT 'ACTIVO'
);

CREATE TABLE medico (
  medico_id SERIAL PRIMARY KEY,
  num_colegiado VARCHAR(30) NOT NULL UNIQUE,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  especialidad_id INT NOT NULL REFERENCES especialidad(especialidad_id),
  estado VARCHAR(10) DEFAULT 'ACTIVO'
);

CREATE TABLE cita (
  cita_id SERIAL PRIMARY KEY,
  paciente_id INT NOT NULL REFERENCES paciente(paciente_id),
  medico_id INT NOT NULL REFERENCES medico(medico_id),
  fecha_cita DATE NOT NULL,
  hora_inicio TIME NOT NULL,
  hora_fin TIME NOT NULL,
  estado VARCHAR(15) DEFAULT 'SOLICITADA',
  codigo_confirmacion VARCHAR(20) UNIQUE,
  fecha_creacion TIMESTAMP DEFAULT NOW()
);

-- Índice para búsqueda por médico y fecha
CREATE INDEX idx_cita_medico_fecha ON cita(medico_id, fecha_cita);
```

## 4.6 Conclusiones del Proyecto

### Logros Alcanzados

- Se desarrolló un análisis completo de la problemática del agendamiento médico en clínicas privadas de la República Dominicana, identificando problemas concretos tanto del lado del paciente como de la institución.
- Se diseñó un modelo de datos relacional normalizado (3FN) con 20 entidades que cubre todos los procesos del negocio: citas, historial clínico, facturación y reportes.
- Se documentaron 12 casos de uso con especificaciones completas de flujo principal, flujos alternativos y excepciones.

- Se diseñó una arquitectura técnica robusta basada en una API REST con autenticación JWT, arquitectura monolítica modular y base de datos PostgreSQL.
- Se elaboró un análisis financiero completo que demuestra la viabilidad del proyecto con un ROI del 181.3% en tres años y un período de recuperación de 7 meses.

### Lecciones Aprendidas

- La fase de análisis de requerimientos es fundamental: una correcta identificación de los actores y sus necesidades desde el inicio previene cambios costosos en etapas posteriores del diseño.
- La normalización del modelo de datos (3FN) facilita la mantenibilidad del sistema y previene anomalías de actualización, aunque requiere joins adicionales en consultas complejas.
- Las decisiones arquitectónicas deben estar justificadas por el contexto del proyecto (tamaño del equipo, escala esperada, presupuesto) y no por tendencias tecnológicas.
- La documentación técnica estructurada facilita la comunicación entre todos los actores del proyecto y sirve como base para la implementación y el mantenimiento futuro.

### Trabajo Futuro

- Implementación real del sistema con el stack tecnológico definido (Node.js + React + PostgreSQL).
- Desarrollo del módulo de telemedicina con videollamadas integradas.
- Integración con sistemas de aseguradoras locales para validación de cobertura en tiempo real.
- Desarrollo de aplicación móvil nativa (iOS y Android) con notificaciones push.
- Implementación de inteligencia artificial para sugerencia de especialistas según síntomas reportados.

# ANEXOS

## Anexo A: Glosario de Términos

| Término | Definición |
|---|---|
| API REST | Interfaz de programación de aplicaciones basada en el protocolo HTTP, que permite la comunicación entre el frontend y el backend del sistema. |
| CIE-10 | Clasificación Internacional de Enfermedades, décima revisión. Estándar internacional para codificación de diagnósticos médicos. |
| JWT | JSON Web Token. Estándar para la transmisión segura de información entre partes como objeto JSON firmado digitalmente. |
| No-show | Término para referirse a un paciente que no asiste a su cita médica sin haberla cancelado previamente. |
| 3FN | Tercera Forma Normal. Nivel de normalización en bases de datos relacionales que elimina dependencias transitivas entre atributos. |
| Redis | Sistema de almacenamiento de datos en memoria de código abierto, utilizado como caché y base de datos de sesiones. |
| ACID | Atomicidad, Consistencia, Aislamiento y Durabilidad. Propiedades que garantizan la fiabilidad de las transacciones en bases de datos. |
| Docker | Plataforma de contenedores que permite empaquetar y ejecutar aplicaciones en entornos aislados y reproducibles. |

## Anexo B: Referencias Bibliográficas


- Pressman, R. S. (2014). Ingeniería del Software: Un Enfoque Práctico (7.a ed.). McGraw-Hill.
- Sommerville, I. (2011). Ingeniería de Software (9.a ed.). Pearson Education.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The Unified Modeling Language Reference Manual (2.a ed.). Addison-Wesley.
- Date, C. J. (2013). An Introduction to Database Systems (8.a ed.). Addison-Wesley.
- Ley 172-13 sobre Protección de Datos de Carácter Personal. República Dominicana, 2013.


## Anexo C: Estructura del Repositorio GitHub

*El proyecto sigue la siguiente estructura de repositorio obligatoria:*

```
MediConnect-Hito4/
├── docs/
│   ├── hito1/ ← vision.pdf, stakeholders, requerimientos.pdf
│   ├── hito2/ ← diagramas-uml/, mer/, diccionario/, informe-avance.pdf
│   ├── hito3/ ← diagramas-estructura/, prototipo/, analisis-costos.pdf
│   └── hito4/ ← documento-final.pdf, presentacion.pptx, enlaces.txt
│
├── diagramas/
│   ├── casos-de-uso/    (archivos .drawio, .xml)
│   ├── actividades/
```

```
│   ├── secuencia/
│   ├── clases/
│   └── mer/             (archivos .mwb, .drawio)
│
├── prototipo/
│   ├── figma-enlace.txt
│   └── capturas/
│
├── README.md
└── .gitignore
```